

INF01151 - Sistemas Operacionais II

Trabalho Prático - Parte 2

Henry R. Piceni¹, João P. F. Pereira¹, Pedro H. F. Fleck¹, Luís F. L. França¹

¹Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

1. Introdução

Este relatório detalha a implementação de um sistema distribuído com tolerância a falhas, desenvolvido para a disciplina *INF01151 - Sistemas Operacionais II*. O foco do trabalho é a aplicação de uma estratégia de replicação passiva [Schneider 1990], onde um servidor primário processa as operações e as propaga de forma síncrona aos backups, e a utilização do algoritmo *Bully* [Garcia-Molina 1982] para a eleição de um novo líder em caso de falha. Nas seções seguintes, serão discutidos em detalhe como a replicação passiva e o algoritmo de eleição foram implementados. Adicionalmente, o documento abordará o ambiente de testes utilizado e os problemas encontrados durante o desenvolvimento.

2. Primeira Parte do Trabalho

A primeira parte do trabalho consistiu na implementação de um serviço de sincronização de arquivos semelhante ao *Dropbox*, utilizando *threads*, *sockets* TCP e mecanismos de sincronização como *mutex* e semáforos. O sistema foi desenvolvido em C++ para ambientes *Unix* e incluiu um servidor capaz de gerenciar múltiplos usuários e dispositivos simultaneamente, garantindo a consistência dos dados através de sincronização automática do diretório `sync_dir`. O servidor utiliza *threads* para atender clientes concorrentemente, enquanto o cliente monitora alterações locais e as propaga para o servidor e outros dispositivos. Foram implementados comandos básicos como `upload`, `download`, `delete` e listagem de arquivos, além de um mecanismo para evitar loops infinitos durante a sincronização.

3. Ambiente de Testes

Os testes foram realizados em um ambiente isolado utilizando contêiner Docker baseado na imagem oficial do Ubuntu 22.04. O contêiner foi executado em um MacBook equipado com chip Apple M2 e 8 GB de memória RAM, rodando o sistema operacional macOS Sequoia (15.3.1). Dentro do contêiner, foram instaladas as ferramentas essenciais de compilação, incluindo o g++ (versão 11.4.0), make (versão 4.3), e cmake (versão 3.22.1), além de dependências como `libpthread-stubs0-dev`, `git`, `vim` e `net-tools`.

O código-fonte do projeto foi copiado para o diretório de trabalho `/app` dentro do contêiner, conforme especificado no `Dockerfile`. A compilação foi realizada a partir desse diretório com os comandos `make clean & make`. É importante observar que o contêiner utiliza o kernel fornecido pelo sistema operacional hospedeiro (macOS), mas todo o ambiente de compilação e execução está restrito ao sistema Ubuntu dentro do Docker, garantindo portabilidade e reprodutibilidade nos testes.

O ambiente de testes foi orquestrado por meio do Docker Compose. Foram definidos dois serviços principais: um servidor e um cliente, ambos construídos a partir

do mesmo contexto Docker localizado no diretório do projeto. O serviço de servidor é responsável por iniciar a aplicação principal, escutando em uma porta específica que é exposta e mapeada diretamente para o host, permitindo o acesso externo ao sistema em execução. O projeto do host é montado como volume compartilhado dentro dos contêineres, o que facilita o desenvolvimento iterativo e garante que alterações no código estejam imediatamente disponíveis no ambiente de testes.

O serviço de cliente é configurado para ser executado em modo interativo, com terminal alocado e entrada padrão aberta, permitindo a execução manual de comandos dentro do contêiner. Essa configuração é particularmente útil para depuração e testes pontuais. Ambos os serviços estão conectados a uma rede virtual interna, gerenciada pelo Docker com driver do tipo bridge, permitindo comunicação direta entre os contêineres por nome de serviço, o que simula de forma realista um ambiente cliente-servidor.

Essa infraestrutura proporciona um ambiente flexível e reproduzível para o desenvolvimento e testes de aplicações distribuídas, combinando automação na execução com a possibilidade de intervenção manual quando necessário.

4. Desenvolvimento

Conforme a figura 1, retirada da especificação do trabalho, a arquitetura do sistema segue o modelo de replicação passiva, com um servidor primário responsável por processar operações e propagar atualizações de forma síncrona para os servidores de backup. Os clientes interagem com o sistema por meio de um *frontend* que encaminha as requisições ao primário. Em caso de falha do primário, o sistema utiliza o algoritmo *Bully* para eleger um novo líder entre os servidores disponíveis, garantindo continuidade e consistência do serviço. Essa abordagem assegura tolerância a falhas com alta disponibilidade.

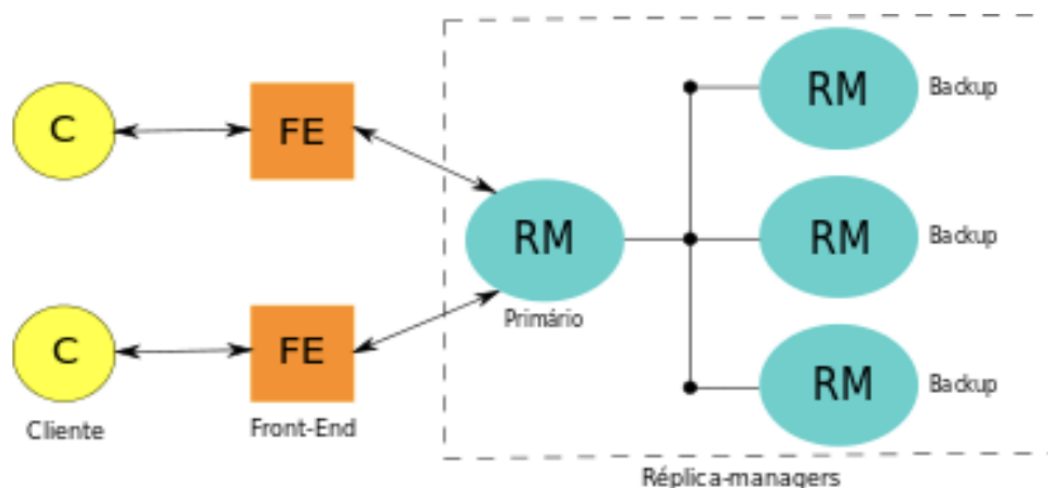


Figura 1. Enter Caption

4.1. Replicação Passiva

A replicação passiva é um mecanismo fundamental para garantir a disponibilidade e a consistência dos dados em sistemas distribuídos. Nessa abordagem, todas as operações de

escrita, como *uploads*, deleções ou modificações de arquivos são inicialmente processadas por um servidor primário. O cliente, ao realizar uma operação, envia sua requisição ao *frontend*, que atua como intermediário e encaminha a solicitação ao servidor primário responsável. As subseções seguintes descrevem a implementação da replicação passiva em nossa implementação.

4.1.1. Núcleo de Replicação

Em nosso trabalho, o método `PrimaryServer::replicateOperation` desempenha papel central. Assim que o primário recebe uma operação, ele a registra em seu *log* interno e imediatamente inicia o processo de replicação, enviando a mesma operação para todos os servidores de *backup* ativos. Esse envio é feito de forma síncrona: o primário aguarda explicitamente as confirmações de recebimento e aplicação da operação por parte dos *backups* antes de prosseguir. Essa lógica garante que nenhuma operação seja considerada concluída até que o sistema esteja, de fato, consistente em todos os nós participantes.

Nos servidores de *backup*, o método `BackupServer::applyReplication` é responsável por receber as mensagens de replicação vindas do primário. Ao receber uma operação, o *backup* utiliza o método `BackupServer::processReplicationOperation` para verificar se aquela operação já foi aplicada, evitando duplicidade e garantindo a ordem correta das operações. Caso a operação seja nova, ela é aplicada ao sistema de arquivos local e registrada no log do *backup*. Após a aplicação bem-sucedida, o *backup* envia uma confirmação de volta ao primário, sinalizando que está sincronizado.

Somente após receber todas as confirmações dos *backups*, o primário responde ao *frontend*, que então confirma ao cliente que a operação foi realizada com sucesso. Esse fluxo garante que, mesmo em caso de falha do primário, qualquer *backup* poderá assumir como novo líder sem perda de dados, pois todos os *backups* estarão atualizados até a última operação confirmada.

4.1.2. Sincronização de Estado Completo

Além da replicação de operações individuais, quando um *backup* se conecta ao primário (ou reconecta após falha), ele pode solicitar a sincronização completa do estado. Isso é feito por métodos como `BackupServer::requestStateSynchronization` e `PrimaryServer::sendFullStateTo`, garantindo que *backups* atrasados ou recém-iniciados recebam todas as operações pendentes desde o último ponto aplicado.

4.1.3. Heartbeat e Monitoramento

O primário e os *backups* trocam mensagens de *heartbeat* para monitorar a saúde das conexões. O primário usa métodos como `monitorBackupConnections` para detectar falhas de *backup*, enquanto os *backups* usam `maintainPrimaryConnection` e `sendHeartbeatToPrimary` para monitorar o primário. Isso é fundamental para detectar falhas rapidamente e acionar mecanismos de eleição ou reconexão.

4.1.4. Reconfirmação e Consistência

O primário mantém um *log* de operações e verifica confirmações pendentes com métodos como `ensureConsistency`. Se algum *backup* não confirmou uma operação, o primário pode reenviar operações não confirmadas, garantindo que todos os *backups* fiquem sincronizados mesmo após falhas temporárias de rede.

4.1.5. Gestão Dinâmica de Backups

O primário pode adicionar ou remover *backups* dinamicamente (`addBackupServer`, `removeBackupServer`) e tenta reconectar a *backups* que ficaram indisponíveis (`handleBackupFailure`, `connectToBackup`). Isso permite que o *cluster* se adapte a mudanças de topologia sem perder consistência.

4.1.6. Fila de Replicação e Threads

Tanto o primário quanto os *backups* usam *threads* dedicadas para processar filas de replicação (`processReplicationQueue` no primário, `processIncomingReplication` no *backup*), evitando bloqueios e melhorando a performance do sistema.

4.1.7. Confirmação Parcial e Fator de Replicação

O primário pode considerar uma operação confirmada após receber confirmações de um número mínimo de *backups*, definido pelo `replication_factor` e calculado em `calculateRequiredConfirmations`. Isso permite tolerar falhas parciais sem bloquear o sistema.

4.2. Algoritmo de Eleição

O algoritmo de eleição utilizado neste trabalho é o algoritmo do *Bully*. Esse algoritmo é clássico em sistemas distribuídos para eleição de líder (primário) entre servidores, garantindo que o servidor com maior prioridade (ou maior ID) assuma o papel de líder em caso de falha do primário atual. Foi preferível o algoritmo *Bully* em relação ao algoritmo do Anel pois o grupo decidiu que o *Bully* fazia mais sentido no contexto do caso de estudo.

4.2.1. Detecção de Falha e Disparo da Eleição

A eleição é iniciada quando um servidor detecta a ausência do primário por meio da falta de mensagens de *heartbeat*. O método responsável por disparar o processo de eleição é o `ElectionManager::startElection`. Esse método pode ser chamado diretamente pelo *ClusterManager* quando a falha do líder é percebida.

4.2.2. Descoberta do Servidor com Maior Prioridade

Uma vez disparada a eleição, o sistema precisa identificar qual servidor ativo possui a maior prioridade (ou maior ID, conforme definido no arquivo de configuração). Para isso, é utilizado o método `ElectionManager::findHighestPriorityAlive`, que percorre a lista de servidores ativos e retorna o ID daquele com maior prioridade.

4.2.3. Proclamação do Novo Líder

Se o servidor que iniciou a eleição for o de maior prioridade, ele se proclama novo líder utilizando o método `ElectionManager::becomeLeader`. Esse método atualiza o estado interno do servidor para refletir seu novo papel e prepara o *cluster* para a transição de liderança.

4.2.4. Atualização do Estado do Cluster e Notificação

Após a proclamação do novo líder, o *cluster* precisa ser atualizado para refletir a nova liderança. O método `ClusterManager::promoteServerToPrimary` é chamado para atualizar o estado do *cluster*, e `ClusterManager::notifyPrimaryChange` é utilizado para notificar o frontend e os demais servidores sobre a mudança de liderança.

4.2.5. Gerenciamento da Eleição

O método `ClusterManager::electNewPrimary` serve como um *wrapper* para delegar o processo de eleição ao `ElectionManager`, garantindo que a coordenação da eleição seja feita de forma centralizada e robusta, evitando conflitos e garantindo que sempre o servidor mais prioritário assuma o papel de primário.

5. Problemas encontrados durante o desenvolvimento

Referências

- Garcia-Molina, H. (1982). Elections in a distributed computing system. In *IEEE Transactions on Computers*, volume C-31, pages 48–59.
- Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319.